LiDAR Object Detection and Tracking

A ROS2 node that performs real-time 3D object detection and tracking from LiDAR point clouds using DBSCAN clustering and centroid-based tracking.

Overview

This node processes LiDAR point cloud data to detect and track objects in the environment. It performs preprocessing, clustering, and multi-object tracking with visualization capabilities.

Main Features:

- Point cloud preprocessing (cropping, downsampling, outlier removal)
- Ground plane removal
- DBSCAN-based clustering
- Simple centroid-based object tracking with EMA smoothing
- Color-coded visualization for each tracked object
- Structured object output with metadata

System Architecture

```
LiDAR Sensor → Point Cloud → Preprocessing → Clustering → Tracking → Outputs
                ↓            ↓             ↓           ↓
            /ouster/points  /processed_  /clustered_  /detected_objects
                            points       points       /cluster_markers
```

Topics

Subscribed Topics

Topic	Type	Description
`/ouster/points`	`sensor_msgs/PointCloud2`	Raw point cloud from Ouster LiDAR sensor

Published Topics

Topic	Type	Description	Update Rate
`/processed_points`	`sensor_msgs/PointCloud2`	Preprocessed point cloud (after cropping, downsampling, outlier removal, ground removal)	Same as input
`/clustered_points`	`sensor_msgs/PointCloud2`	Colored point cloud where each cluster has a unique color	Same as input
`/cluster_markers`	`visualization_msgs/MarkerArray`	Visualization markers (text labels + center spheres) for each tracked object	Same as input
`/detected_objects`	`detected_object_msgs/DetectedObjectArray`	Structured object data with ID, position, distance, angle, point count	Same as input

---

Output Visualization

1. Colored Point Clouds (`/clustered_points`)

- Each tracked object is rendered in a unique color
- Colors are consistently assigned to each track ID
- Only shows objects that meet the minimum hit threshold

2. Markers (`/cluster_markers`)

- **Text Markers**: Display track ID above each object (offset +1.0m in Z)
- **Sphere Markers**: 0.4m diameter sphere at the centroid of each object
- Both markers use the same color as their corresponding point cloud
- Short lifetime (0.2s) to prevent stale markers

3. Detected Objects (`/detected_objects`)

Each detected object contains:

- `id`: Unique track ID
- `center`: 3D position (x, y, z) of the object centroid
- `point_count`: Number of points in the cluster

- `distance`: 2D Euclidean distance from sensor (sqrt($x^2 + y^2$))
- `angle_deg`: Angle in degrees from sensor (atan2(y, x))

Parameters

Preprocessing Parameters

Parameter	Type	Default	Description
`crop_min_x`	double	0.0	Minimum X boundary for ROI (meters)
`crop_max_x`	double	0.0	Maximum X boundary for ROI (meters)
`crop_min_y`	double	0.0	Minimum Y boundary for ROI (meters)
`crop_max_y`	double	0.0	Maximum Y boundary for ROI (meters)
`crop_min_z`	double	0.0	Minimum Z boundary for ROI (meters)
`crop_max_z`	double	0.0	Maximum Z boundary for ROI (meters)
`voxel_size`	double	0.0	Voxel grid size for downsampling (meters)
`sor_nb_neighbors`	int	0	Number of neighbors for statistical outlier removal
`sor_std_ratio`	double	0.0	Standard deviation ratio for outlier removal
`ground_z_threshold`	double	0.0	Z-height threshold to remove ground points (meters)

Clustering Parameters

Parameter	Type	Default	Description
`dbscan_eps`	double	0.0	Maximum distance between points in a cluster (meters)
`dbscan_min_points`	int	0	Minimum points required to form a cluster

Tracking Parameters

Parameter	Type	Default	Description
`track_max_distance`	double	0.0	Maximum distance to associate cluster with existing track (meters)
`track_max_age`	int	0	Maximum frames a track can exist without detection before deletion
`track_min_hits`	int	0	Minimum detections required before publishing a track
`ema_alpha`	double	0.0	Exponential moving average smoothing factor (0-1, higher = more responsive)

Processing Pipeline

1. Point Cloud Preprocessing

```
Raw Points → Crop ROI → Voxel Downsample → Statistical Outlier Removal → Ground Removal
```

- **Cropping**: Defines 3D bounding box to focus on area of interest
- **Downsampling**: Reduces point density using voxel grid
- **Outlier Removal**: Removes noise using statistical analysis
- **Ground Removal**: Filters points below Z threshold

2. Clustering (DBSCAN)

- Groups nearby points into clusters representing objects
- `eps`: Maximum distance between points in same cluster
- `min_points`: Minimum cluster size to be considered valid

3. Object Tracking

- **Association**: Matches new clusters to existing tracks using nearest centroid
- **Smoothing**: Uses EMA to smooth centroid position over time

```
new_position = α × measured + (1-α) × previous
```

- **Track Management**:
  - New tracks created for unmatched clusters
  - Track "age" increments when not detected
  - Tracks deleted after exceeding `track_max_age`
  - Only tracks with ≥ `track_min_hits` are published

Critical Information & Troubleshooting

⚠️ Common Issues

1. **No Output on `/detected_objects`**

- Check that `track_min_hits` is set appropriately (try 1-3)
- Verify objects are being detected in `/clustered_points`
- Ensure preprocessing parameters aren't filtering everything out

2. **Too Many False Detections**
   - Increase `dbscan_min_points` to require larger clusters
   - Increase `track_min_hits` to require more consistent detections
   - Tune `sor_nb_neighbors` and `sor_std_ratio` for better noise removal
3. **Objects Not Tracking Properly**
   - Increase `track_max_distance` if objects move quickly
   - Increase `track_max_age` if detections are intermittent
   - Adjust `ema_alpha` (0.3-0.7 typical range)
4. **Performance Issues**
   - Increase `voxel_size` to reduce point density
   - Reduce crop region size
   - Increase `dbscan_eps` to create fewer, larger clusters

🔧 Recommended Parameter Ranges

```yaml
```

```yaml
# Typical urban/indoor environment
crop_min_x: -10.0
crop_max_x: 10.0
crop_min_y: -10.0
crop_max_y: 10.0
crop_min_z: -1.5
crop_max_z: 2.0
voxel_size: 0.05          # 5cm voxels
sor_nb_neighbors: 20
sor_std_ratio: 1.0
ground_z_threshold: -0.3   # Remove ground

dbscan_eps: 0.3           # 30cm clustering distance
dbscan_min_points: 10      # At least 10 points per object

track_max_distance: 2.0    # 2m association distance
track_max_age: 5           # 5 frames before deletion
track_min_hits: 3          # Require 3 consistent detections
ema_alpha: 0.5             # 50% smoothing
```

📊 Understanding the Output

Distance Calculation:

- Uses 2D distance: $\sqrt{x^2 + y^2}$ (ignores Z)
- Useful for ground-based navigation

Angle Calculation:

- Range: -180° to +180°
- 0° = directly in front (positive X)
- 90° = left side (positive Y)
- -90° = right side (negative Y)

Color Assignment:

- Each track ID gets a unique HSV color
- Same ID always has same color across frames
- Colors persist as long as track is active

Dependencies

- ROS2 (tested on Humble/Iron)
- Python 3
- `rclpy`
- `sensor_msgs`
- `visualization_msgs`
- `geometry_msgs`
- `detected_object_msgs` (custom message package)
- `open3d`
- `numpy`
- `sensor_msgs_py`

Running the Node

Basic Launch

```bash
ros2 run <package_name> lidar_object_detector.py
```

With Parameters

```bash
```

```
ros2 run <package_name> lidar_object_detector.py --ros-args \
  -p crop_min_x:=-10.0 -p crop_max_x:=10.0 \
  -p crop_min_y:=-10.0 -p crop_max_y:=10.0 \
  -p crop_min_z:=-1.5 -p crop_max_z:=2.0 \
  -p voxel_size:=0.05 \
  -p sor_nb_neighbors:=20 \
  -p sor_std_ratio:=1.0 \
  -p ground_z_threshold:=-0.3 \
  -p dbscan_eps:=0.3 \
  -p dbscan_min_points:=10 \
  -p track_max_distance:=2.0 \
  -p track_max_age:=5 \
  -p track_min_hits:=3 \
  -p ema_alpha:=0.5
```

Visualization in RViz2

Add the following displays:

1. **PointCloud2** → `/processed_points` (white)
2. **PointCloud2** → `/clustered_points` (RGB color)
3. **MarkerArray** → `/cluster_markers`

Set Fixed Frame to your LiDAR frame (e.g., `ouster_lidar`, `base_link`)

Message Definitions

You'll need the custom message package with:

DetectedObject.msg:

```
int32 id
geometry_msgs/Vector3 center
int32 point_count
float64 distance
float64 angle_deg
```

DetectedObjectArray.msg:

```
std_msgs/Header header
DetectedObject[] objects
```

Performance Considerations

- **Point cloud size**: Preprocessing significantly reduces points (10x-100x reduction typical)
- **Tracking overhead**: O(n×m) where n=clusters, m=tracks (negligible for <50 objects)
- **EMA smoothing**: Reduces jitter but adds slight lag (tune `ema_alpha`)
- **Marker lifetime**: Set to 0.2s to prevent accumulation; adjust based on your frame rate

---

Future Improvements

- [ ] Bounding box estimation for each cluster
- [ ] Velocity estimation using Kalman filters
- [ ] Classification (pedestrian, vehicle, static object)
- [ ] Multi-sensor fusion
- [ ] Configurable visualization colors
- [ ] Dynamic reconfigure for runtime parameter tuning
- [ ] Object size/dimension estimation

License

[Add your license here]

Author

[Add your name/contact here]